

Solution Requirements

Date	26 June 2026
Team ID	
Project Name	Smart Lender
Maximum Marks	4 Marks

Functional and Non-Functional Requirements:

Functional and non-functional requirements define the features and quality standards of a software system. Functional requirements describe what the application should do, while non-functional requirements define how efficiently, securely, and reliably it should perform.

For the **Smart Lender – Loan Eligibility Prediction System**, these requirements ensure the application accurately predicts loan eligibility, provides a user-friendly interface, delivers quick responses, and securely handles applicant data for a smooth and reliable user experience.

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	<i>Not applicable in current scope. The application is currently designed as an open, public-facing utility tool, allowing instant access to the prediction engine without requiring an account or login.</i>	Low
2	Authorization levels	<i>Not applicable in current scope. All users currently share the same access level, giving everyone the ability to input financial metrics and instantly view the predicted loan outcome.</i>	Low
3	External interfaces	<i>The Flask backend seamlessly interfaces with the trained Machine Learning pipeline (loading rdf.pkl and scale.pkl). Future state includes</i>	High

		<i>connecting to external Credit Bureau APIs.</i>	
4	Transactions processing	<i>The system captures 11 financial metrics (Income, Dependents, Loan Amount, etc.) from the frontend form and securely transmits them to the Python backend for real-time XGBoost ML processing.</i>	High
5	Reporting	<i>The system generates a dynamic Results Dashboard that displays the Final Decision (Approved/Rejected), the precise Confidence Probability (%), and categorizes the user into a Risk Band.</i>	Medium
6	Business rules, etc.	<i>Strict validation logic is enforced: incomes and loan amounts must be > 0, and categorical fields (Property Area, Education) must perfectly match the normalized schema expected by the ML model.</i>	High
7	Compliance to laws or regulations	<i>The system must comply with data privacy laws (e.g., GDPR/financial regulations) by not permanently storing sensitive personal identifying information (PII) after the prediction is made unless authorized.</i>	High
8	Error Handling & Validation	<i>The system must gracefully intercept empty fields, invalid text, or missing data submitted by the user. Instead of crashing the server, it must safely</i>	High

		<i>default to numeric values or return a user-friendly error message on the screen.</i>	
--	--	---	--

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	<i>The Machine Learning inference (scaling the inputs and calculating the XGBoost probabilities) must execute virtually instantaneously.</i>	<i>Returns the final ML prediction and loads the result page in < 1.5 seconds.</i>
2	Scalability	<i>The application must handle multiple prediction requests efficiently by keeping the lightweight .pkl ML models pre-loaded in the server's memory rather than reloading them every time.</i>	<i>Supports concurrent prediction processing without exceeding 512MB of server RAM.</i>
3	Security & Data Privacy	<i>The system must protect user privacy by operating as a stateless utility. It processes the financial data in memory to make the prediction but intentionally avoids saving it.</i>	<i>0% of the applicant's financial form data is stored in a database or written to disk after the session ends.</i>
4	Reliability & Availability	<i>The Flask backend must remain highly resilient. It must remain available to serve predictions and not shut down even if a user submits unexpected characters or invalid data.</i>	<i>0 backend server crashes occur during invalid or incomplete form submissions.</i>
5	Usability & Accessibility	<i>The web frontend must provide visually intuitive feedback, utilizing the Premium Glassmorphism design and easily understandable Risk</i>	<i>The UI layout is responsive across screen sizes and requires zero user training to successfully submit an application.</i>

		<i>Bands (Low, Moderate, High).</i>	
6	<i>Other</i>	<i>Maintainability:</i> <i>The Machine Learning Python logic must remain completely decoupled from the Web UI, ensuring developers can edit the web design without breaking the AI.</i>	<i>Updating the HTML/CSS frontend requires absolutely 0 changes to the Python ML pipeline or model files.</i>